



上海财经大学

SHANGHAI UNIVERSITY OF FINANCE AND ECONOMICS

《数据科学导论》项目报告

课程名称： 数据科学导论

项目主题： 基于 XGBoost 的成都二手房价格预测研究

学生姓名： 章泽俊

任课老师： 袁浩禹

院 系： 信息管理与工程学院

专 业： 数据科学与大数据技术

2026 年 6 月

摘要

二手房价格预测是房地产数据分析中的经典回归问题，对于购房者、房产中介及城市规划设计均具有重要的参考价值。本研究以成都市为研究对象，设计并实现了一套完整的二手房价格预测流程：首先使用 Selenium 与 BeautifulSoup 对安居客（Anjuke）成都二手房列表页进行动态渲染与结构化抽取，获取房源标题、总价、单价、面积、户型、朝向、楼层、建造年份、小区、城区、商圈、地址、标签等 16 个字段；随后通过高德地图开放 API 对房源地址进行地理编码，并补充最近地铁站距离、1500 m 内地铁站数量、2000 m 内学校数量等 POI（Point of Interest）特征；数据以 MySQL 关系型数据库进行持久化存储；在数据清洗与探索性分析阶段，对原始字段进行数值化提取、缺失值填补、异常值过滤，并绘制房价分布、区域小提琴图、特征相关性热力图、地铁距离与房价关系图等；在机器学习阶段，构建了包含交互特征、非线性变换、分箱特征与目标编码在内的高级特征体系，对比了 Ridge、Lasso、ElasticNet、Random Forest、Extra Trees、Gradient Boosting、XGBoost、LightGBM 八种模型，最终选用 XGBoost 并通过网格搜索进行粗调与细调两阶段超参数优化。实验结果表明，调优后的 XGBoost 在测试集上取得了 $R^2 = 0.9094$ 、RMSE = 25.85 万元、MAE = 14.07 万元的成绩，5 折交叉验证 R^2 均值达到 0.8391；基于 SHAP 的可解释性分析显示，“is_luxury”（大面积且多地铁组合）、面积相关特征、区域目标编码是影响房价的核心因素。最后，本研究基于 Flask 开发了可视化看板，提供首页数据概览、单套房价预测、房源搜索与数据分析四大功能模块，实现了从数据采集、存储、建模到应用部署的完整数据科学闭环。

关键词：二手房价格预测；XGBoost；网络爬虫；POI 特征增强；数据可视化；SHAP 可解释性

目录

1	绪论	1
1.1	研究背景	1
1.2	研究目的与意义	1
1.3	主要研究内容	1
2	数据采集：对象、技术与反爬	2
2.1	数据来源与目标	2
2.2	安居客页面结构分析	3
2.3	爬虫方案与架构	3
2.4	反爬策略	5
2.5	高德地图 POI 数据补充	5
2.6	采集规模	6
3	数据存储	7
3.1	数据库表设计	7
3.2	Python 数据库访问与数据流	8
4	数据预处理与探索性分析	9
4.1	数据合并与特征提取	9
4.2	缺失值与异常值处理	9
4.3	探索性数据分析	9
4.4	本章小结	10
5	数据挖掘与机器学习	12
5.1	任务定义与评价指标	12
5.2	特征工程	12
5.3	基线模型对比	13
5.4	XGBoost 超参数调优	14
5.5	模型评估	15
5.6	SHAP 可解释性分析	17
6	结果可视化与看板	21
6.1	静态可视化	21
6.2	Flask Web 看板	21
7	结果分析与讨论	22

8 总结与展望	23
8.1 工作总结	23
8.2 未来工作	23
A 附录 A：爬虫核心代码	24
B 附录 B：特征工程核心代码	25

1 绪论

1.1 研究背景

随着中国城镇化进程的持续推进，房地产市场始终是居民资产配置与城市经济发展的核心议题之一。成都作为西部地区的国家中心城市，近年来人口持续流入，住房需求旺盛，二手房交易规模不断扩大。然而，二手房价格受到地段、面积、房龄、交通便利性、教育资源、小区品质等多重因素的综合影响，具有显著的非线性和异质性，传统的比较法或经验估价难以快速、准确地给出量化参考。

与此同时，互联网上积累了大量结构化的房产挂牌信息，安居客、链家、贝壳找房等平台每日更新海量房源数据。如何自动、高效地获取这些数据，并结合地理信息、交通设施、教育资源等外部 POI 特征，构建高精度的房价预测模型，已成为数据科学与房地产经济交叉研究的重要方向。

1.2 研究目的与意义

本研究旨在通过完整的数据科学工作流，实现以下目标：

1. **数据采集自动化**：针对安居客成都二手房列表页，设计稳定、可复用的网络爬虫，自动获取多区域、多页房源信息，并将数据持久化到 MySQL 数据库。
2. **外部特征增强**：调用高德地图 API，对房源地址进行地理编码，并补充地铁站距离、周边地铁与学校数量等 POI 特征，提升模型对“地段”因素的刻画能力。
3. **高精度价格预测**：基于 XGBoost 梯度提升树模型，结合丰富的特征工程与超参数调优，构建准确的房价预测模型。
4. **结果可解释与可视化**：利用 SHAP 分析关键特征对预测的影响，并通过 Flask Web 看板展示数据概览、预测功能和房源检索，增强成果的实用性与交互性。

本研究的意义在于：一方面，验证了从网络数据采集、地理信息增强到机器学习建模的完整技术链路；另一方面，为成都二手房市场提供了可解释、可交互的价格参考工具，具有一定的学术价值与应用价值。

1.3 主要研究内容

本研究的主要工作包括：

- **数据层**：使用 `db_create.py` 初始化 MySQL 数据库与数据表；使用 `chengdu_ershoufang_spider` 爬取安居客房源；使用 `gaode_enrich_poi_api.py` 调用高德 API 补充 POI 特征。

- **分析层**：在 `main.ipynb` 中完成数据读取、合并、清洗、特征工程、模型训练、超参数调优、模型评估与 SHAP 解释。
- **应用层**：在 `house-website/` 中构建 Flask Web 可视化看板，支持数据概览、房价预测、房源搜索与数据分析。

2 数据采集：对象、技术与反爬

2.1 数据来源与目标

本研究选择安居客成都二手房频道 (<https://chengdu.anjuke.com/sale/>) 作为主要数据来源。安居客是国内领先的房产信息服务平台，其二手房列表页包含丰富的结构化字段，适合用于价格预测建模。研究目标是从列表页抽取如表 1 所示的核心字段。

表 1: 安居客房源核心字段

字段	含义	说明
<code>title</code>	房源标题	房源描述性标题
<code>total_price</code>	总价	含“万”的原始字符串
<code>unit_price</code>	单价	元/m ²
<code>area_size</code>	建筑面积	含“m ² ”的字符串
<code>layout_full</code>	户型	如“3 室 2 厅 1 卫”
<code>orientation</code>	朝向	如“南北”
<code>floor</code>	楼层	如“中楼层”
<code>build_year</code>	建造年份	如“2015 年建”
<code>community</code>	小区名称	—
<code>district</code>	城区	如“高新区”
<code>sub_district</code>	商圈	如“金融城”
<code>street</code>	详细地址	—
<code>tags</code>	特色标签	如“近地铁”
<code>area_code</code>	区域代码	爬虫自定义
<code>detail_url</code>	详情页链接	—
<code>source</code>	数据来源	固定为“anjuke”

爬取区域覆盖成都市 16 个主要区县，包括：武侯区、高新区、成华区、金牛区、锦江区、青羊区、双流区、龙泉驿区、温江区、郫都区、新都区、天府新区、青白江区、都江堰市、崇州市、简阳市。每个区域按页码递增抓取，单页约 60 条房源。

2.2 安居客页面结构分析

安居客二手房列表页为服务端渲染的 HTML 页面，每页约展示 60 套房源，列表容器使用 `div.property` 类选择器标识。房源卡片内部结构如下：

- 标题: `h3.property-content-title-name`
- 总价: `span.property-price-total-num + span.property-price-total-text`
- 单价: `p.property-price-average`
- 户型/面积/朝向/楼层/建造年份: `div.property-content-info` 下的多个 `p.property-content-info-text`
- 小区名称: `p.property-content-info-comm-name`
- 地址: `p.property-content-info-comm-address` 下的多个 `span`
- 标签: `span.property-content-info-tag`
- 详情链接: `a.property-ex`

由于列表页不存在复杂的 SPA (Single Page Application) 动态加载，采用 Selenium 进行页面渲染后即可使用 BeautifulSoup 解析 HTML，兼顾了稳定性与开发效率。

2.3 爬虫方案与架构

爬虫实现于 `chengdu_ershoufang_spider.py`，核心类 `ChengduAnjukeSpider` 的设计要点如下：

1. **浏览器驱动**: 使用 Selenium + ChromeDriver，开启 headless 模式，设置窗口大小、User-Agent，并移除 `navigator.webdriver` 属性以规避部分反爬检测。
2. **区域遍历**: 按预设的 16 个区域代码循环抓取，每个区域最多抓取 50 页。
3. **断点续传与反爬控制**: 安居客存在较强的反爬机制，高频请求容易被识别并封禁 IP，因此爬虫采取“低频次、分批次”策略：每次只抓取 2 个区域，并在批次之间间隔数小时，同时借助 `crawl_progress` 表记录每个区域已抓取的页码，既避免触发风控，也支持中断后从断点继续。
4. **数据去重**: 使用 INSERT IGNORE 基于主键避免重复入库。
5. **错误重试**: 每页最多重试 3 次，若触发验证码则跳过该区域。

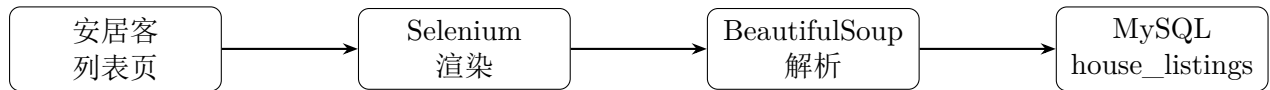


图 1: 爬虫数据流架构

整体数据流可概括为：安居客列表页 → Selenium 渲染 → BeautifulSoup 解析 → MySQL 持久化，如图 1 所示。

代码 1 展示了页面加载与 DOM 抽取的核心逻辑。

```
1 def _get_page(self, url):
2     for attempt in range(3):
3         try:
4             self.driver.get(url)
5             WebDriverWait(self.driver, 15).until(
6                 EC.presence_of_all_elements_located(
7                     (By.CLASS_NAME, 'property')))
8             time.sleep(random.uniform(2, 4))
9             html = self.driver.page_source
10            if '验证' not in html and 'captcha' not in html.lower():
11                return html
12            except Exception:
13                pass
14            time.sleep(10 * (attempt + 1))
15        return None
16
17 @staticmethod
18 def parse_list(html):
19     soup = BeautifulSoup(html, 'html.parser')
20     cards = soup.find_all('div', class_='property')
21     houses = []
22     for card in cards:
23         try:
24             house = {}
25             t = card.find('h3', class_='property-content-title-name')
26             house['title'] = t.get('title', '').strip() if t else ''
27             # ... 其他字段解析 ...
28             houses.append(house)
29         except Exception:
30             continue
31     return houses
```


Listing 1: 页面加载与解析（节选自 chengdu_ershoufang_spider.py）

2.4 反爬策略

为防止被安居客反爬机制拦截，爬虫采取了表 2 所列策略，所有手段均以合规、低压力为前提。

表 2: 反爬/稳定性问题与应对

问题	应对
动态渲染	Selenium 模拟浏览器渲染 + 等待选择器
请求频率限制	每页随机休眠 8–15 秒，分批抓取（每次 2 个区域）
User-Agent 检测	模拟最新版 Chrome/Edge 浏览器
自动化特征检测	通过 CDP 将 navigator.webdriver 置为 undefined
验证码拦截	检测“验证”/“captcha”关键字，触发则终止当前区域
任务中断	crawl_progress 表记录进度，支持断点续爬

2.5 高德地图 POI 数据补充

房源列表页虽然包含地址文本，但缺乏经纬度及周边设施等量化信息。为增强模型对“地段”因素的刻画，本研究调用高德地图开放 API，实现以下功能（见 gaode_enrich_poi_api.py）：

1. **地理编码**：通过 /v3/geocode/geo 将地址转换为经纬度。
2. **最近地铁站距离**：通过 /v3/place/around 搜索“地铁站”，计算房源到最近地铁站的球面距离（Haversine 公式）。
3. **周边地铁数量**：统计 1500 m 范围内的地铁站数量。
4. **周边学校数量**：统计 2000 m 范围内的学校数量。

代码 2 展示了 POI 补充的核心函数。

```
1 def geocode(address):
2     url = 'https://restapi.amap.com/v3/geocode/geo'
3     params = {'key': AMAP_KEY, 'address': f'成都市{address}', 'city':
4               '成都'}
5     r = requests.get(url, params=params, timeout=10).json()
6     if r.get('status') == '1' and r.get('geocodes'):
```

```
6         loc = r['geocodes'][0]['location']
7         lng, lat = map(float, loc.split(','))
8         return lng, lat
9     return None, None
10
11 def nearest_subway(lng, lat):
12     url = 'https://restapi.amap.com/v3/place/around'
13     params = {
14         'key': AMAP_KEY, 'location': f'{lng},{lat}',
15         'keywords': '地铁站', 'radius': 5000, 'offset': 1, 'page': 1
16     }
17     r = requests.get(url, params=params, timeout=10).json()
18     if r.get('status') == '1' and r.get('pois'):
19         loc = r['pois'][0].get('location', '')
20         if loc:
21             plng, plat = map(float, loc.split(','))
22             return int(haversine(lng, lat, plng, plat))
23     return -1
24
25 def count_poi(lng, lat, keyword, radius):
26     url = 'https://restapi.amap.com/v3/place/around'
27     params = {
28         'key': AMAP_KEY, 'location': f'{lng},{lat}',
29         'keywords': keyword, 'radius': radius, 'offset': 25, 'page':
30         1
31     }
32     r = requests.get(url, params=params, timeout=10).json()
33     if r.get('status') == '1':
34         return len(r.get('pois', []))
35     return 0
```

Listing 2: 高德 POI 特征补充（节选自 gaode_enrich_poi_api.py）

2.6 采集规模

经过多轮爬取与 POI 补充，本研究最终获得：

- 房源核心数据：约 10,758 条；
- POI 补充数据：约 2,041 条（按小区 + 地址去重）；

- 覆盖区域：16 个区县。

合并 POI 后，约 10,753 条房源具备完整的面积、价格及区域信息，可用于后续建模。

3 数据存储

3.1 数据库表设计

数据库 `chengdu_house` 包含三张核心表：

`house_listings`——房源核心数据表

表 3: 房源核心数据表结构

字段	类型	说明
<code>id</code>	<code>INT AUTO_INCREMENT</code>	主键
<code>title</code>	<code>VARCHAR(500)</code>	房源标题
<code>total_price</code>	<code>VARCHAR(50)</code>	总价（原始字符串）
<code>unit_price</code>	<code>VARCHAR(50)</code>	单价
<code>area_size</code>	<code>VARCHAR(50)</code>	建筑面积
<code>layout_full</code>	<code>VARCHAR(50)</code>	户型
<code>orientation</code>	<code>VARCHAR(20)</code>	朝向
<code>floor</code>	<code>VARCHAR(100)</code>	楼层
<code>build_year</code>	<code>VARCHAR(20)</code>	建造年份
<code>community</code>	<code>VARCHAR(200)</code>	小区名称
<code>district</code>	<code>VARCHAR(50)</code>	城区
<code>sub_district</code>	<code>VARCHAR(50)</code>	商圈
<code>street</code>	<code>VARCHAR(300)</code>	详细地址
<code>tags</code>	<code>VARCHAR(300)</code>	特色标签
<code>area_code</code>	<code>VARCHAR(30)</code>	区域代码
<code>detail_url</code>	<code>VARCHAR(800)</code>	详情页链接
<code>source</code>	<code>VARCHAR(20)</code>	数据来源
<code>created_at</code>	<code>TIMESTAMP</code>	创建时间

表 4: 爬虫进度表结构

字段	类型	说明
area_code	VARCHAR(30)	区域代码
source	VARCHAR(20)	来源
last_page	INT	已抓最大页码
total_count	INT	累计入库数
updated_at	TIMESTAMP	更新时间

`crawl_progress`——爬虫进度表

`house_poi`——房源 POI 特征表

表 5: 房源 POI 特征表结构

字段	类型	说明
community	VARCHAR(200)	小区名称
street	VARCHAR(300)	详细地址
lng	DECIMAL(10,7)	经度
lat	DECIMAL(10,7)	纬度
nearest_subway_dist	INT	最近地铁站距离（米）
subway_count	INT	1500 m 内地铁站数
school_count	INT	2000 m 内学校数
poi_updated_at	TIMESTAMP	更新时间

3.2 Python 数据库访问与数据流

本研究使用 `pymysql` 进行数据库连接与操作。`db_create.py` 负责初始化数据库与表结构，爬虫与 POI 补充模块均复用同一套 `DB_CONFIG` 配置。在建模阶段，`main.ipynb` 直接通过 `pandas.read_sql` 将 MySQL 数据读取为 `DataFrame`，实现数据库与分析环境的无缝衔接。

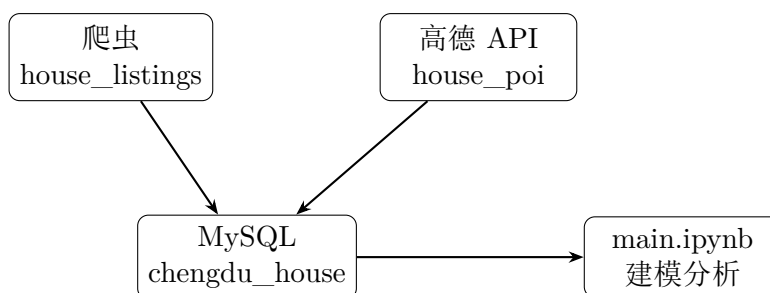


图 2: 数据存储与使用流程

4 数据预处理与探索性分析

4.1 数据合并与特征提取

在 `main.ipynb` 中，首先读取 `house_listings` 与 `house_poi` 两张表，并以 `community` 和 `street` 为键进行左连接。由于 POI 数据基于小区 + 地址去重，部分房源可能存在 POI 缺失，对此采用填充策略：最近地铁站距离缺失填充 -1，地铁与学校数量缺失填充 0，经纬度缺失填充 0。

原始字段大多为字符串类型，需进行数值化提取：

- 总价：price_num 从 total_price 中提取数字（万元）。
- 面积：area_num 从 area_size 中提取数字（m²）。
- 户型：从 layout_full 中提取室、厅、卫数量，分别记为 room_count、hall_count、toilet_count。
- 房龄：build_year_num 为年份数字，house_age = 2026 - build_year_num。
- 地铁标签：has_subway 表示 tags 中是否包含“地铁”。

4.2 缺失值与异常值处理

缺失值处理：总价、面积等关键字段缺失的记录直接删除；POI 字段缺失按上述业务语义填充。

异常值处理：为保证建模质量，对价格和面积进行合理范围过滤：

- 总价范围：10-2000 万元；
- 面积范围：20-500 m²；
- 删除总价或面积为空的记录。

过滤后剩余约 10,753 条有效样本，样本量充足且分布合理。

4.3 探索性数据分析

房价、面积、房龄分布

清洗后样本的总价、面积与房龄分布显示：总价呈现明显的右偏分布，大部分房源集中在 100-300 万元区间；面积分布相对集中，60-120 m² 占比最高；房龄分布显示，成都二手房以 5-20 年房龄为主。

区域房价小提琴图

图 3 选取房源量最多的 8 个城区绘制小提琴图。可以看出，高新区、锦江区、青羊区等核心城区房价均值显著高于双流区、龙泉驿区等外围区域，地段因素对房价的影响非常明显。

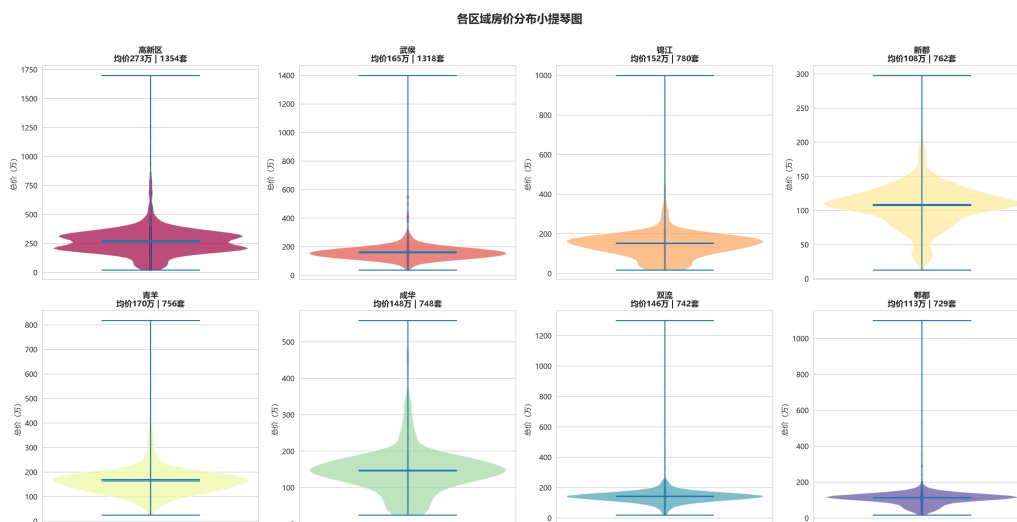


图 3: 各区域房价分布小提琴图

特征相关性热力图

图 4 展示了价格与核心数值特征之间的 Spearman 相关系数。面积、区域目标编码、户型房间数与总价呈较强正相关；房龄、最近地铁站距离与总价呈负相关。

地铁距离与房价关系

图 5 展示了不同地铁距离区间对应的房价箱线图与均值柱状图。整体趋势表明，距离地铁站越近，房价均值越高，“地铁房”溢价效应明显。

4.4 本章小结

本章完成了从原始字符串到数值特征、从单表到多表合并、从异常值过滤到多维可视化的完整预处理流程。核心发现包括：

- 成都二手房总价呈右偏长尾，需关注中位数与分位数；
- 核心城区与外围城区房价差异显著，“地段”是强解释变量；
- 面积、房龄、地铁距离与总价存在单调关系，为后续特征工程提供方向。

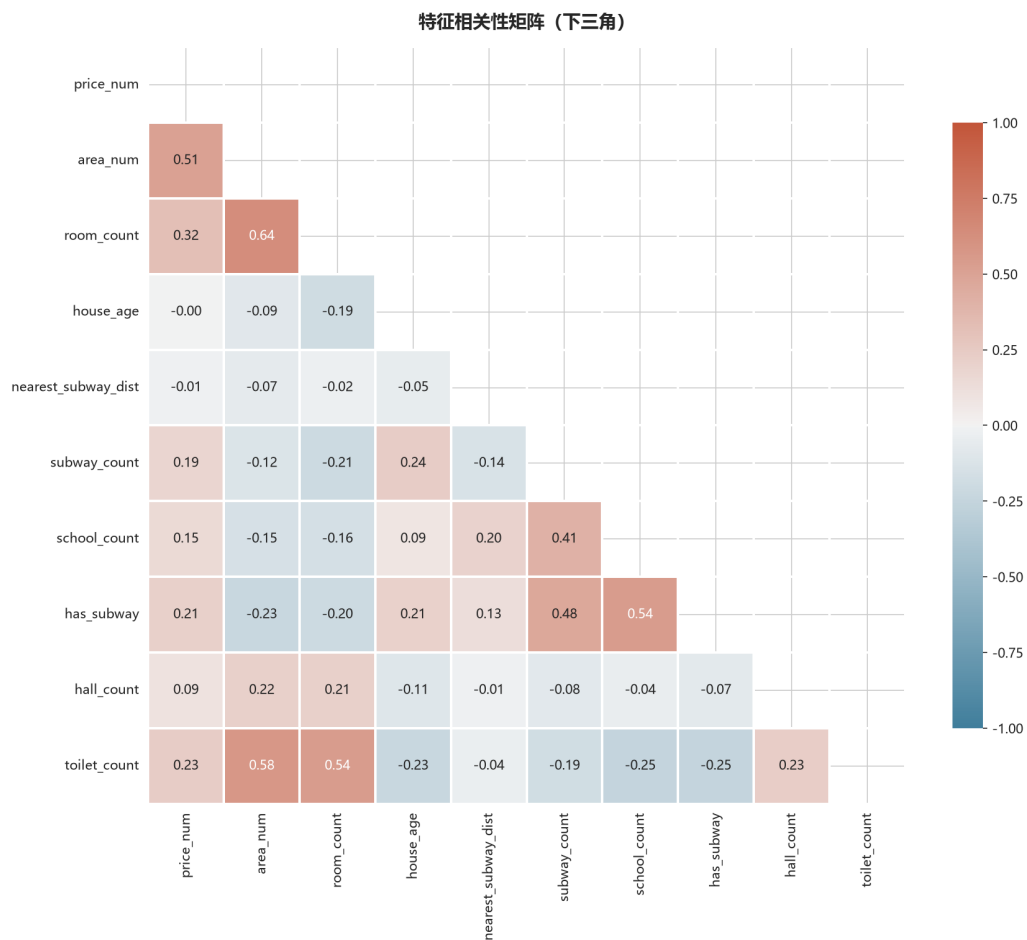


图 4：特征相关性热力图

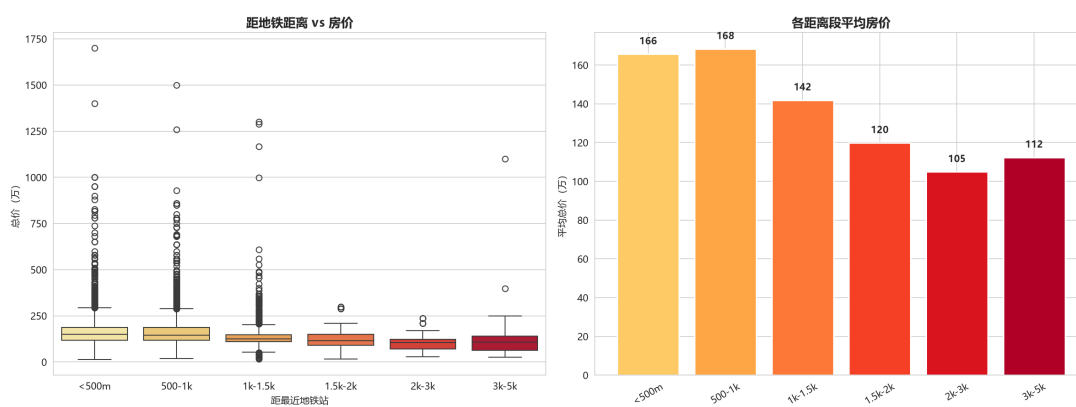


图 5：地铁距离与房价关系

5 数据挖掘与机器学习

5.1 任务定义与评价指标

本研究将房价预测定义为回归任务：给定房源特征向量 $\mathbf{x}$ ，预测其总价 y （万元）。采用的评价指标包括：

- 决定系数 R^2 ：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- 均方根误差 RMSE：

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- 平均绝对误差 MAE：

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

数据按 8:2 划分为训练集与测试集，`random_state=42` 保证可复现性。同时采用 5 折交叉验证评估模型稳定性。

5.2 特征工程

在基础特征之上，本研究构建了丰富的高级特征，将原始 14 个基础特征扩展为 35 个建模特征。

交互特征

- `area_x_subway` = 面积 × 地铁数量
- `area_x_school` = 面积 × 学校数量
- `age_x_area` = 房龄 × 面积
- `age_x_subwaydist` = 房龄 × (地铁距离 + 1)
- `subway_x_school` = 地铁数量 × 学校数量
- `room_density` = 房间数 / 面积
- `area_per_room` = 面积 / 房间数

非线性变换

- `log_area`、`area_squared`、`sqrt_area`
- `inv_subway_dist` = $1 / (\text{地铁距离} + 100)$
- `age_squared`、`log_age`

分箱特征

- `area_bin`: 面积分箱 (< 60、60–90、90–120、120–150、150–200、200+)
- `age_bin`: 房龄分箱 (新房、5–10 年、10–15 年、15–20 年、20–30 年、30 年 +)
- `subway_bin`: 地铁距离分箱 (< 500 m、500–1 k、1 k–2 k、2 k–5 k、无地铁)

目标编码 (Target Encoding)

为防止高基数类别特征 (如 `district`、`community`、`street`) 直接 one-hot 导致维度灾难, 本研究采用带平滑的目标编码:

$$\hat{y}_{category} = \frac{n_c \cdot \bar{y}_c + \lambda \cdot \bar{y}_{global}}{n_c + \lambda}$$

其中 n_c 为该类别样本数, $\bar{y}_c$ 为该类别目标均值, $\bar{y}_{global}$ 为全局均值, $\lambda = 10$ 为平滑系数。编码对象包括城区、面积分箱、房龄分箱、地铁距离分箱。

组合特征

- `total_rooms` = 室 + 厅
- `toilet_ratio` = 卫 / (室 + 1)
- `is_luxury` = 面积 > 150 m² 且地铁数量 ≥ 2
- `old_and_far` = 房龄 > 20 年且地铁距离 > 2000 m

5.3 基线模型对比

为在多种回归模型中选取最适合本任务的模型, 本研究对比了 Ridge、Lasso、ElasticNet、Random Forest、Extra Trees、Gradient Boosting、XGBoost、LightGBM 共 8 种回归模型。所有模型均在统一的数据集与特征体系下训练, 并采用 5 折交叉验证评估稳定性。结果如表 6 所示。

从表 6 可以看出:

表 6: 基线回归模型对比 (测试集 / 5 折交叉验证)

模型	RMSE (万)	MAE (万)	测试集 R^2	CV R^2 均值	CV R^2 标准差
XGBoost	26.00	15.33	0.9083	0.8409	0.0223
Extra Trees	27.49	16.94	0.8975	0.8162	0.0273
Gradient Boosting	28.06	17.07	0.8932	0.8383	0.0229
Random Forest	31.07	17.52	0.8691	0.8175	0.0342
LightGBM	32.44	19.95	0.8573	0.7846	0.0188
Ridge	49.56	30.51	0.6669	0.6326	0.0190
Lasso	50.59	31.18	0.6529	0.6310	0.0089
ElasticNet	50.71	31.28	0.6513	0.6303	0.0126

- 树模型整体显著优于线性模型，说明房价与特征之间存在复杂的非线性关系。
- 在 8 种模型中，XGBoost 在测试集 R^2 与交叉验证稳定性上均表现最优，因此被确定为最终模型。
- 线性模型 (Ridge/Lasso/ElasticNet) R^2 仅约 0.65，反映出简单线性假设难以刻画房价的非线性结构。

5.4 XGBoost 超参数调优

为获得更优的 XGBoost 模型，本研究采用两阶段网格搜索。

粗调阶段

主要搜索 `n_estimators` 与 `learning_rate`:

表 7: XGBoost 粗调搜索空间

参数	搜索范围
<code>n_estimators</code>	100, 300, 500
<code>learning_rate</code>	0.05, 0.1, 0.2
<code>max_depth</code>	6
<code>subsample / colsample_bytree</code>	0.8

粗调最优结果: `n_estimators=500`, `learning_rate=0.1`, 3 折 CV $R^2 = 0.8367$ 。

细调阶段

在粗调基础上固定 `n_estimators` 与 `learning_rate`, 精细搜索结构正则化参数:

细调最优参数为: `max_depth=8`, `subsample=0.9`, `colsample_bytree=0.7`, `reg_alpha=0.1`, `reg_lambda=2`。

表 8: XGBoost 细调搜索空间

参数	搜索范围
max_depth	4, 6, 8, 10
subsample	0.7, 0.8, 0.9
colsample_bytree	0.7, 0.8, 0.9
reg_alpha	0, 0.1, 1
reg_lambda	1, 2, 5

调优后性能

在测试集上，调优后的 XGBoost 取得：

- $R^2 = 0.9094$
- RMSE = 25.85 万元
- MAE = 14.07 万元
- 5 折 CV $R^2 = 0.8391 \pm 0.0223$

5.5 模型评估

学习曲线

图 6 展示了 XGBoost 的学习曲线。随着训练样本增加，训练集 R^2 缓慢下降，验证集 R^2 逐步上升并趋于稳定，两者最终差距较小，说明模型未出现严重过拟合。

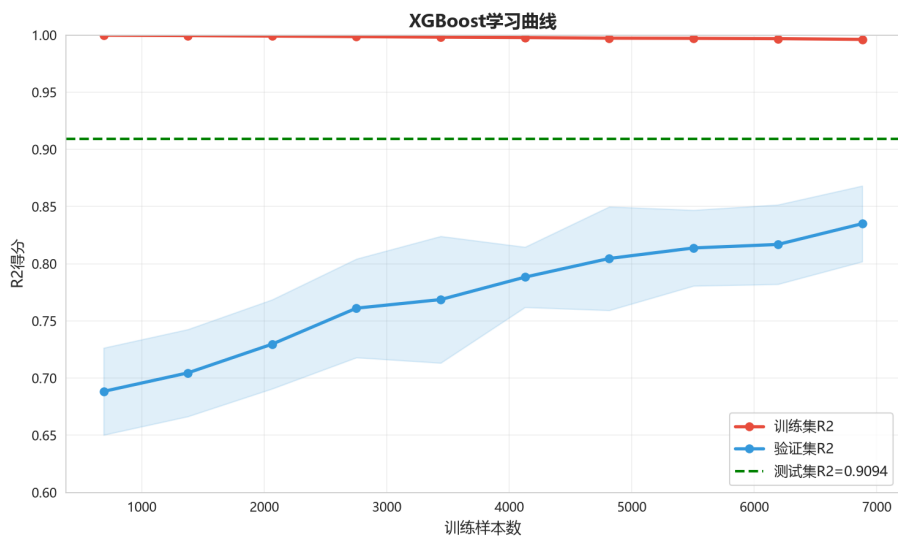


图 6: XGBoost 学习曲线

最终评估图

图 7 从四个维度评估模型：

- **真实值 vs 预测值散点图**：点密集分布在对角线附近，说明预测整体准确。
- **残差分布直方图**：残差近似服从均值为 0 的正态分布。
- **残差图**：残差随机分布在 0 附近，无明显异方差模式。
- **分价格段 MAE**：高价段（500 万 +）误差绝对值较大，符合房价预测中高价房波动更大的普遍规律。

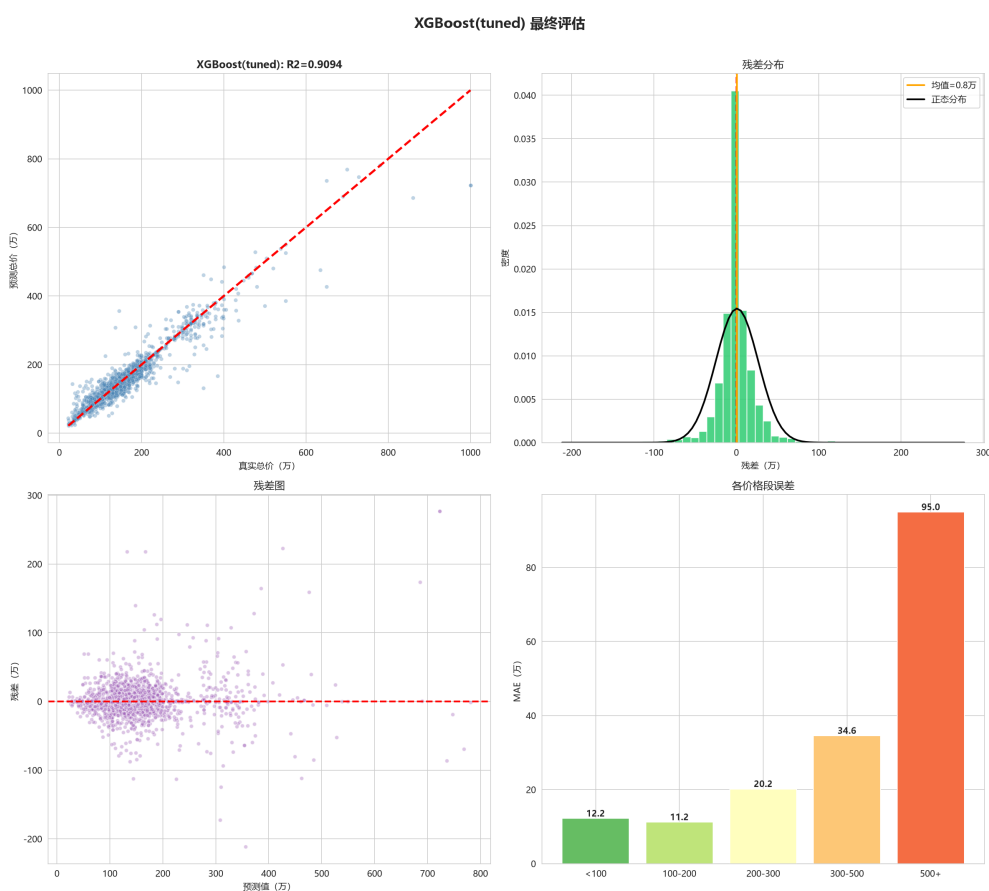


图 7: XGBoost 最终评估

特征重要性

图 8 同时展示了 XGBoost 的 Gain 重要性与 Permutation 重要性。两种方法均显示 `is_luxury`（大面积且多地铁）、`sqrt_area`、`district_te`（城区目标编码）、`area_bin_te`（面积分箱目标编码）等特征对模型贡献最大。

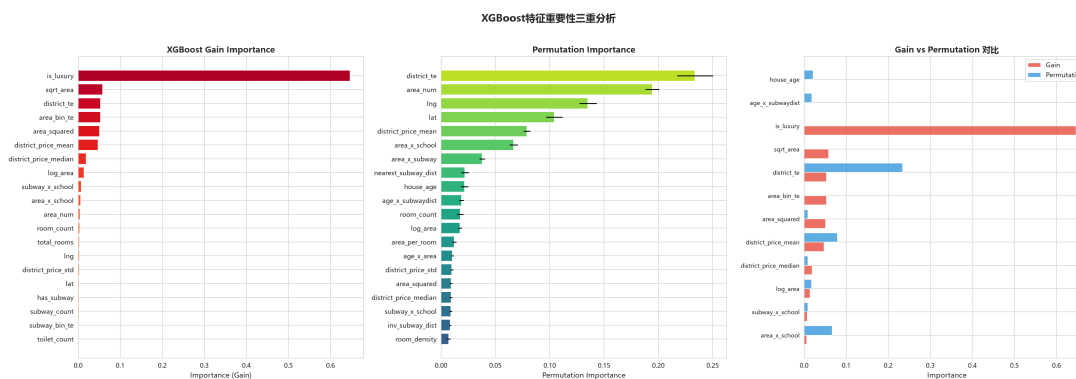


图 8: XGBoost 特征重要性

5.6 SHAP 可解释性分析

为进一步解释模型预测机制，本研究使用 SHAP（SHapley Additive exPlanations）从全局与局部两个层面剖析 XGBoost 模型的决策逻辑。SHAP 基于博弈论中的 Shapley 值，将每个特征对单次预测的贡献量化，使得复杂的树集成模型变得可解释。

全局解释

图 9 的 SHAP Summary Plot 展示了 Top 20 特征对预测值的影响方向与强度。颜色越红表示特征值越高，越蓝表示越低；横轴为 SHAP 值（对预测价格的正向或负向贡献）。从全局视角可以得出以下结论：

- **面积相关特征占据主导地位：** `sqrt_area`、`area_num`、`area_bin_te` 等特征不仅 SHAP 值绝对值大，而且呈现明显的单调趋势——面积越大，对房价的推高效应越强。这说明在控制了地段与配套之后，面积仍是决定二手房总价的核心物理属性。
- **地段因素具有强区分度：** `district_te`（城区目标编码）与 `area_bin_te`（面积分箱目标编码）的高 SHAP 值表明，不同城区之间的价格梯度显著。高新区、锦江区等核心区域的目标编码较高，对房价产生显著正向贡献；而外围区县则相反。
- **“豪宅”标签形成价格跃升：** `is_luxury=1`（大面积且周边至少有 2 个地铁站）的样本普遍获得大幅正向 SHAP 值，说明大面积与多地铁的组合会产生协同溢价，模型成功捕捉了这种非线性交互。
- **房龄与地铁距离呈负向作用：** `house_age`、`nearest_subway_dist` 的蓝色点集中在左侧负 SHAP 值区域，意味着房龄越老、离地铁越远，房价越容易被拉低；而 `subway_count`、`school_count` 则主要表现为正向贡献，验证了交通与教育配套的溢价效应。

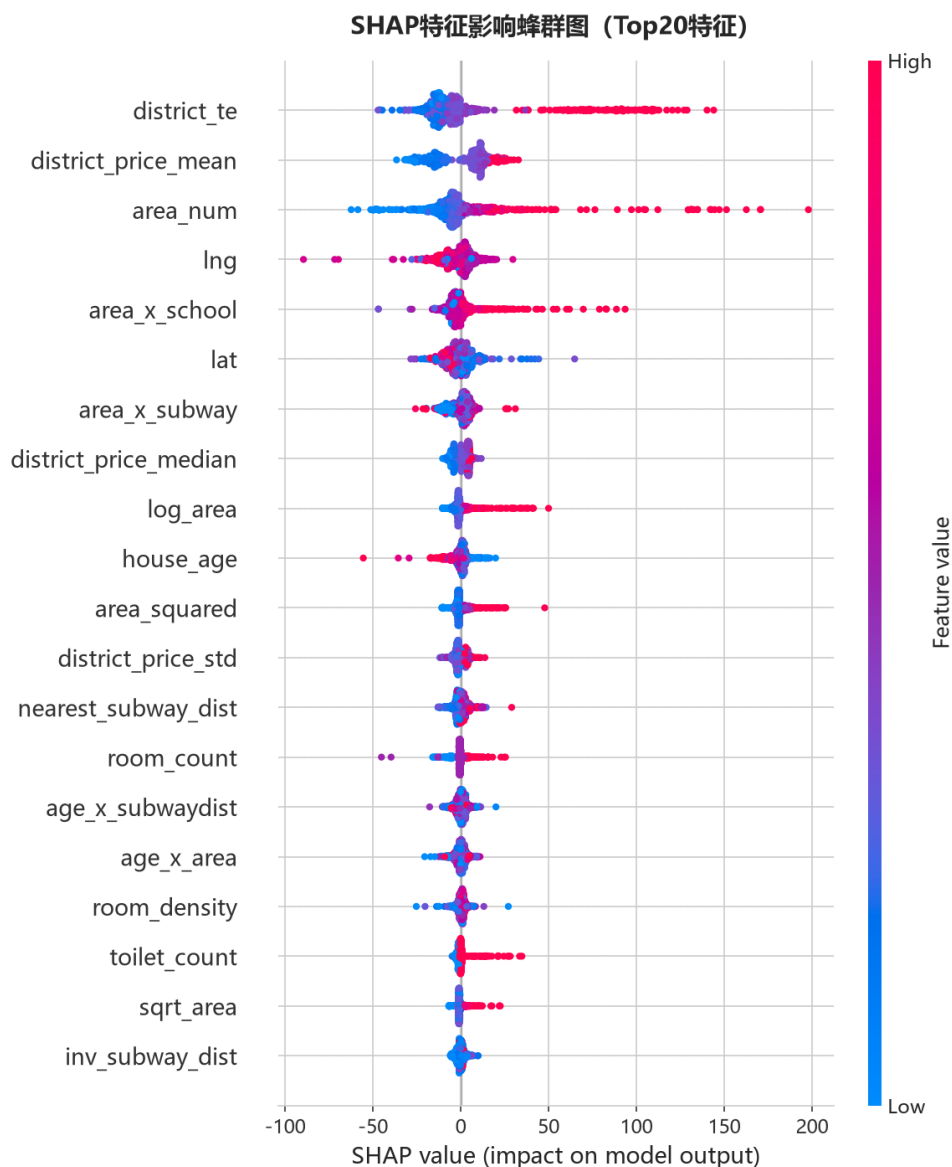


图 9: SHAP 特征影响蜂群图

局部解释

全局分析揭示了模型的“平均”行为，而局部解释则展示模型在单个样本上的具体决策路径。图 10 和图 11 分别展示了测试集中预测最高价与最低价的两个典型房源的 SHAP Waterfall 图，横轴为每个特征对最终预测价格的贡献，红色箭头表示正向推动，蓝色箭头表示负向拉动。

最高房价样本：该房源被预测为高价，主要受以下因素驱动：所属城区目标编码极高（核心地段）、建筑面积超过 150 m² 且周边地铁站数量不少于 2 个（满足 is_luxury 条件）、房龄相对较新。Waterfall 图显示，这些因素从基准价格（训练集平均价格）出发，将预测值显著推高至最终高价。即使存在少量负向特征（如略远的最近地铁站），也不足以抵消核心地段的溢价。

最低房价样本：该房源被预测为低价，核心原因在于所属城区目标编码低（外围区县）、面积较小、房龄较老，且周边缺乏地铁与学校配套。Waterfall 图中，这些负向特征层层叠加，将基准价格压低，最终形成一个较低的预测总价。局部解释表明，模型在极端样本上依然遵循“地段 + 面积 + 配套”的定价逻辑，没有出现反常的决策路径。

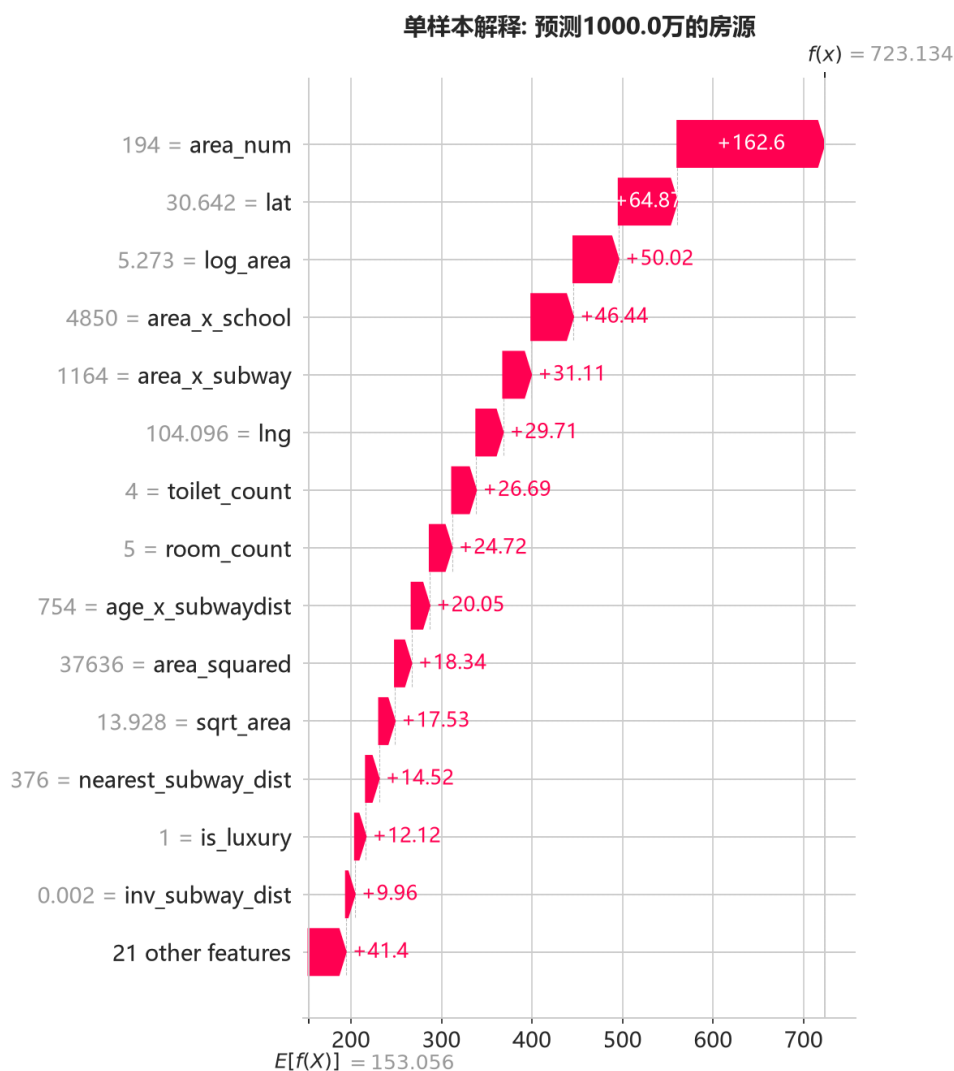


图 10: SHAP Waterfall——最高房价样本

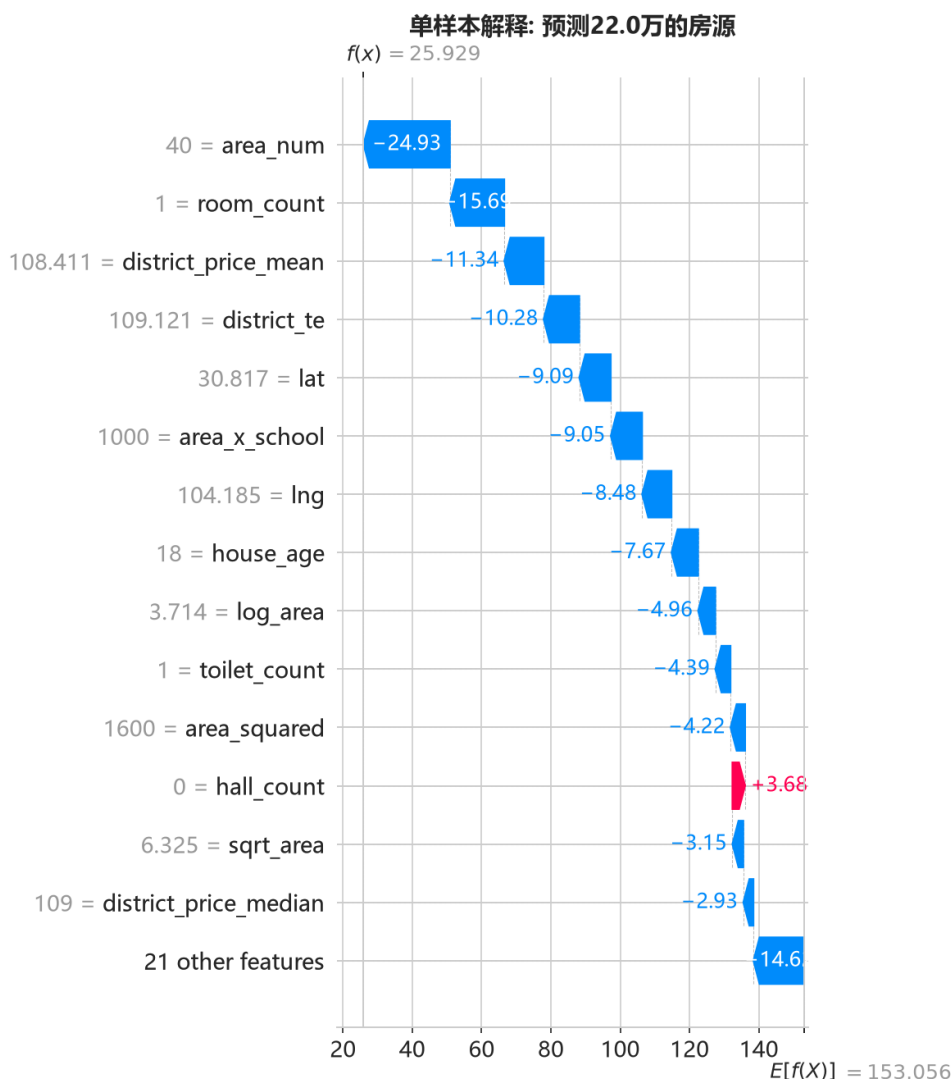


图 11: SHAP Waterfall——最低房价样本

热力图

图 12 展示了前 100 个测试样本在 Top 20 特征上的 SHAP 值热力图，每一列代表一个样本，每一行代表一个特征，颜色越红表示该特征对该样本的预测贡献越正向，越蓝则越负向。热力图进一步印证了全局结论：`is_luxury`、`sqrt_area`、`district_te` 等特征在大多数样本上均呈现稳定的正向或负向作用，说明模型的关注重点具有一致性，未出现严重的样本特异性偏差。

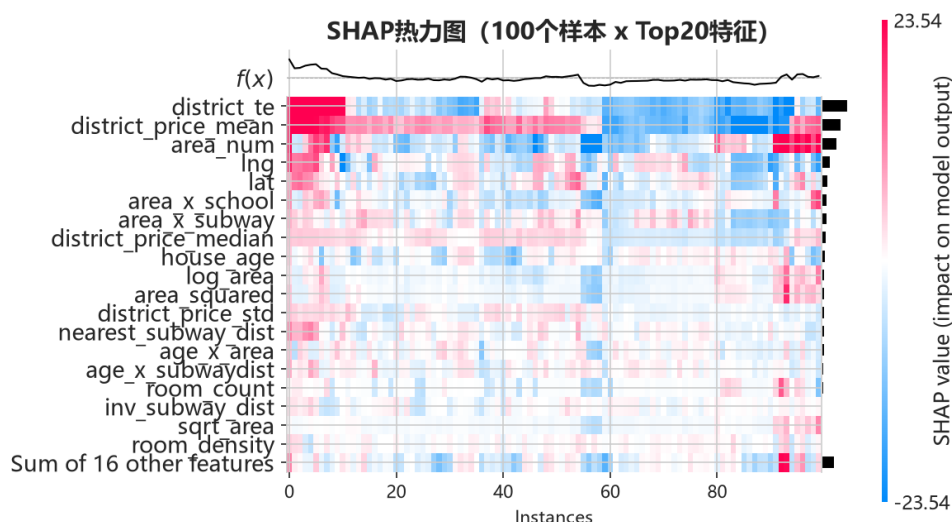


图 12: SHAP 热力图

6 结果可视化与看板

6.1 静态可视化

在前文探索性分析与模型评估中，已使用 Python 的 pandas、matplotlib 与 seaborn 绘制了房价分布、区域小提琴图、相关性热力图、地铁距离分析、学习曲线、残差诊断、特征重要性与 SHAP 解释图。这些静态图表的作用是：把“模型黑盒”转化为可解释的统计证据，支撑结论的可信度。

6.2 Flask Web 看板

为将模型成果转化为可交互的应用，本研究基于 Flask 开发了“成都二手房数据科学平台”（house-website/app.py）。应用采用 MVC 架构：

- **Model:** 加载 `xgboost_model.pkl`、`scaler.pkl`、`enc_map.pkl`、`feature_names.pkl`。
- **View:** 使用 Jinja2 模板渲染首页（`index.html`）、预测（`predict.html`）、搜索（`search.html`）、数据分析（`analytics.html`）四个页面。
- **Controller:** 通过 RESTful API 与 MySQL 交互，提供数据查询与预测服务。

功能模块

- **首页 (Home):** 展示平台核心指标：总房源数、覆盖城区数、平均总价，并提供四大功能入口。

- **房价预测 (Predict):** 用户可选择城区、输入面积、室/厅/卫数量、建造年份、是否有地铁、最近地铁距离、周边地铁与学校数量，系统调用 XGBoost 模型返回预测总价与单价。若模型未加载，则退回到基于区域基价、房龄系数与地铁系数的规则预测。
- **房源搜索 (Search):** 支持按城区、户型、价格区间筛选房源，结果以卡片形式分页展示。
- **数据分析 (Analytics):** 使用 Chart.js 展示区域房价均值柱状图、户型分布饼图、热门小区排行榜等统计图表，并展示模型关键指标 (R^2 、RMSE、MAE)。

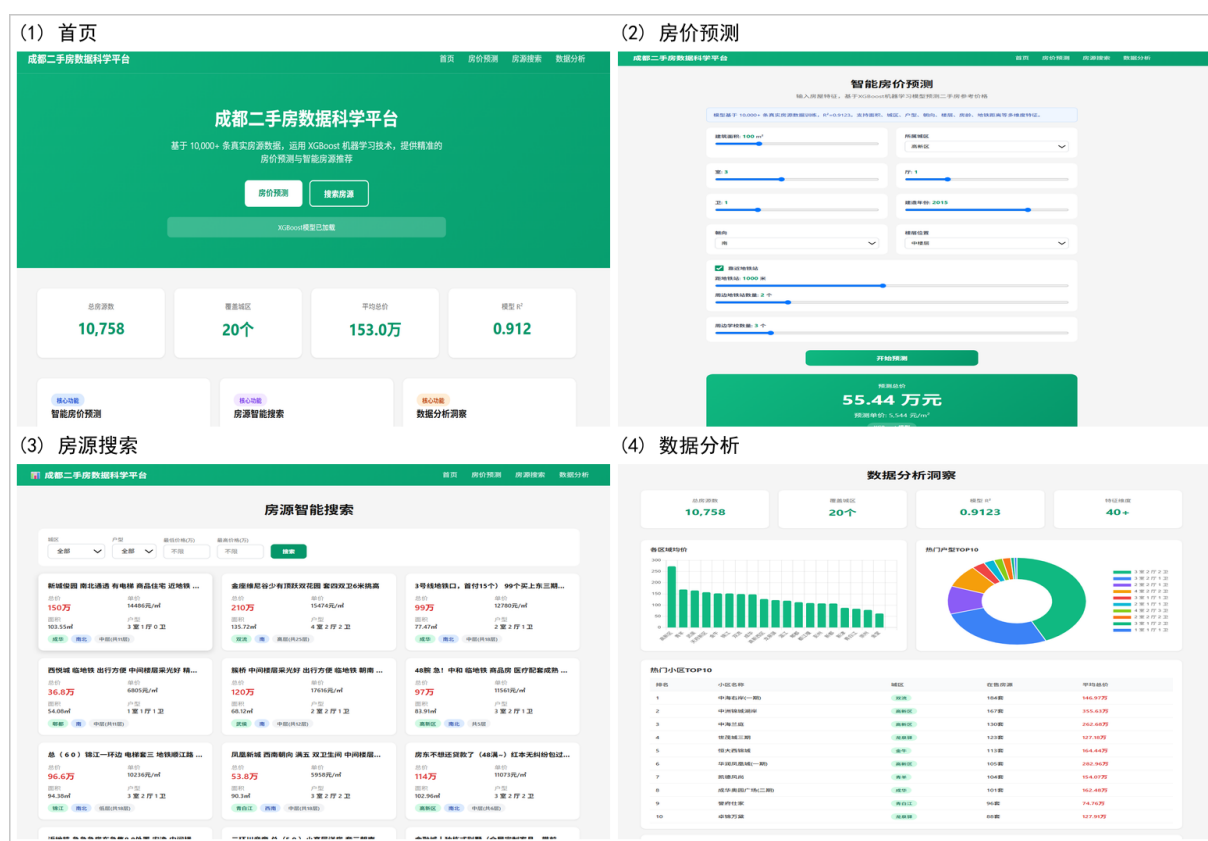


图 13: 成都二手房数据科学平台主要功能界面

7 结果分析与讨论

本研究构建的成都二手房价格预测模型在测试集上达到了 $R^2 = 0.9094$ 的精度，说明所采用的特征体系能够有效解释房价变异的 90% 以上。以下对关键结果进行讨论：

1. **地段因素主导价格:** 城区目标编码 `district_te` 与面积分箱目标编码 `area_bin_te` 均为 Top 特征，印证了成都房地产市场的“地段为王”规律。高新区、锦江区、青羊区等核心区域房价显著高于外围区县。

2. **面积是最核心的物理属性：**`sqrt_area`、`area_squared`、`area_num` 等面积相关特征重要性极高，说明在其他条件相近时，面积是决定总价的最直接因素。
3. **地铁与教育配套具有溢价效应：**地铁数量、最近地铁站距离、学校数量及其交互特征均对模型有正向贡献。地铁数量多、距离近的房源通常具有更高的价格。
4. **非线性模型优势明显：**相比线性模型，XGBoost 能够自动捕捉特征间复杂的交互与非线性关系， R^2 提升超过 0.24，验证了梯度提升树在房价预测任务中的适用性。
5. **模型仍存在的局限：**
 - 高价房（500 万 +）预测误差相对较大，可能与高价房样本较少、个性化因素（装修、景观、学位）未纳入有关。
 - POI 特征仅覆盖了地铁站与学校，未包含医院、商场、公园等其他设施。
 - 爬取的数据为挂牌价而非真实成交价，可能与实际交易价格存在偏差。

8 总结与展望

8.1 工作总结

本研究以成都二手房为研究对象，完成了从数据采集、存储、清洗、特征工程、机器学习建模到可视化应用部署的完整数据科学项目。主要贡献包括：

1. 设计并实现了基于 Selenium + BeautifulSoup 的安居客爬虫，获取约 10,758 条房源数据。
2. 调用高德地图 API 补充了经纬度、地铁距离、周边地铁与学校数量等 POI 特征。
3. 使用 MySQL 构建了规范化的数据存储方案，支持断点续传与数据去重。
4. 通过丰富的特征工程与多模型对比，最终以 XGBoost 实现 $R^2 = 0.9094$ 的房价预测精度。
5. 基于 SHAP 完成了模型的全局与局部可解释性分析。
6. 开发了 Flask 可视化看板，实现数据概览、房价预测、房源搜索与数据分析功能。

8.2 未来工作

未来可从以下方向继续优化：

1. **数据层面**：爬取房源详情页，补充装修情况、楼层类型、建筑结构、绿化率、物业费更细粒度特征；引入真实成交数据以减少挂牌价偏差。
2. **特征层面**：扩展 POI 类型（医院、商场、公园、写字楼），计算到 CBD 距离、学区等级等；引入时间序列特征，分析房价趋势。
3. **模型层面**：尝试 CatBoost、深度学习模型（如 TabNet、FT-Transformer），并引入模型融合（Stacking）进一步提升精度。
4. **应用层面**：优化看板交互体验，加入地图可视化、房价热力图、房源推荐等功能。

A 附录 A：爬虫核心代码

```
1 def run(self, pages_per_area=10, areas_per_batch=2):
2     progress = self._load_progress()
3     remaining = [a for a in self.areas if progress.get(a, 0) < 50]
4     remaining.sort(key=lambda a: progress.get(a, 0))
5     if not remaining:
6         print("所有区域已爬完")
7         return
8
9     batch = remaining[:areas_per_batch]
10    self.driver = self._create_driver()
11    try:
12        self.driver.get('https://chengdu.anjuke.com/')
13        time.sleep(random.uniform(3, 5))
14        for area in batch:
15            start = progress.get(area, 0) + 1
16            end = min(start + pages_per_area - 1, 50)
17            for page in range(start, end + 1):
18                url = (f'https://chengdu.anjuke.com/sale/{area}/'
19                     if page == 1 else
20                     f'https://chengdu.anjuke.com/sale/{area}/p{
21                         page}/')
22                html = self._get_page(url)
23                if html is None:
24                    break
25                houses = self.parse_list(html)
26                for h in houses:
```

```
26         h['area_code'] = area
27         n = self._save_to_db(houses)
28         self._save_progress(area, page, n)
29         time.sleep(random.uniform(8, 15))
30     finally:
31         self.driver.quit()
```

Listing 3: 爬虫批量抓取主循环（节选自 chengdu_ershoufang_spider.py）

B 附录 B：特征工程核心代码

```
1 def create_features(input_data):
2     area = float(input_data.get('area', 100))
3     district = input_data.get('district', '锦江区')
4     rooms = float(input_data.get('rooms', 3))
5     halls = float(input_data.get('halls', 1))
6     toilets = float(input_data.get('toilets', 1))
7     build_year = float(input_data.get('build_year', 2015))
8     has_subway = 1 if input_data.get('has_subway', False) else 0
9     nearest_subway_dist = float(input_data.get('nearest_subway_dist',
10         2000))
11     subway_count = float(input_data.get('subway_count', 0))
12     school_count = float(input_data.get('school_count', 0))
13
14     house_age = 2026 - build_year
15     row = { ... }
16     F = pd.DataFrame([row])
17
18     # 交互特征
19     F['area_x_subway'] = F['area_num'] * F['subway_count']
20     F['area_x_school'] = F['area_num'] * F['school_count']
21     F['age_x_area'] = F['house_age'] * F['area_num']
22     F['subway_x_school'] = F['subway_count'] * F['school_count']
23
24     # 非线性变换
25     F['log_area'] = np.log1p(F['area_num'])
26     F['sqrt_area'] = np.sqrt(F['area_num'])
27     F['inv_subway_dist'] = 1 / (F['nearest_subway_dist'] + 100)
```

```
27
28 # 目标编码
29 gm = enc_map.get('global_mean', 200.0)
30 F['district_te'] = F['district'].map(enc_map.get('district_te',
31     {})).fillna(gm)
32
33 # 组合特征
34 F['is_luxury'] = ((F['area_num'] > 150) & (F['subway_count'] >=
35     2)).astype(int)
36 F['old_and_far'] = ((F['house_age'] > 20) &
37     (F['nearest_subway_dist'].fillna(9999) >
38     2000)).astype(int)
39
40 # 对齐特征顺序
41 if feature_names:
42     for c in feature_names:
43         if c not in F.columns:
44             F[c] = 0
45     F = F[feature_names]
46 return F.astype(float)
```

Listing 4: Web 端特征工程与预测（节选自 house-website/app.py）